

Topic 6.6

6.6: Binding Times

At the end of inherit1.cpp, the code

```
p = &me;  
p->print();  
cout << endl;  
  
p = &president;  
p->print();  
cout << endl;
```

produces the output

```
name:Gary J. Blair  
name:Bill Clinton
```

For all functions we have seen so far, the compiler must choose at compile-time which particular version of the function will be invoked. In the above code, the pointer `p` can point to a `Thing`, or to anything derived from `Thing`. Therefore, in determining which version of the `print` function to invoke, the compiler must choose one which will work for *everything* that `p` can possibly point to. The only reasonable choice is to choose the `Thing::print()` function.

We can delay until runtime which particular version of *print* is invoked by declaring it to be a *virtual* function. Declaring a function *virtual* in the base class means it will automatically be virtual in all derived classes.

It is less efficient (but not by much) to invoke *virtual* functions than to invoke non-virtual functions. However, you have considerably more flexibility.

```
// MODULE:      inherit2.cpp  
// PROGRAMMER:  Gary J. Blair  
// LANGUAGE:    C++  
// DATE:       96/04/11  
//  
// PURPOSE:  
//   Illustrate virtual functions.  
//  
  
#include <iostream>  
#include <stdlib.h>  
#include <string.h>
```

```

using namespace std;

class Thing    // Thing is the base of
               // all our objects.
{
    public:
    Thing() { strcpy(name,""); };
    Thing( char * thing_name )
    { strcpy(name,thing_name); };
    virtual void print();

    protected:
    char name[80];
};

void Thing::print()
{
    cout << "name:" << name;
}

class Person : public Thing
{
    public:
    Person();
    Person( char *person_name, int person_age );
    void print();    // Automatically virtual
                    // from base class.

    protected:
    int age;
};

Person::Person() : Thing()
{
    age = -1;
}

Person::Person
( char *person_name, int person_age ) :
    Thing(person_name)
{
    age = person_age;
}

void Person::print()
{
    Thing::print();
    cout << " age: " << age;
}

class Politician : public Person
{
    public:
    Politician( char *person_name, int person_age,
               char *pol_party );
    void print();    // Automatically virtual from
                    // base class.

    protected:
    char party[40];
};

Politician::Politician
( char *person_name, int person_age,
  char *pol_party ):
    Person(person_name,person_age)
{

```

```
        strcpy(party,pol_party);
    }

    void Politician::print()
    {
        Person::print();
        cout << " party:" << party;
    }

int main(void)
{
    Thing      t("Cessna 182");
    Person      me("Gary J. Blair",37);
    Politician  president("Bill Clinton",
46,"Democrat");
    Thing      *p;

    t.print();          cout << "\n";
    me.print();          cout << "\n";
    president.print();  cout << "\n";

    p = &t;
    p->print();
    cout << endl;

    p = &me;
    p->print();
    cout << endl;

    p = &president;
    p->print();
    cout << endl;

    return 0;
}
```

Output from inherit2.cpp:

```
name:Cessna 182
name:Gary J. Blair age: 37
name:Bill Clinton age: 46 party:Democrat
name:Cessna 182
name:Gary J. Blair age: 37
name:Bill Clinton age: 46 party:Democrat
```

Topic 6.7 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-6-dot-7>).